

1. 文件头部与立即执行函数

javascript

```
/** * 多曲线编辑器 - 每个控制点可独立调整斜率（切线控制柄） * 采用 Hermite 插值，转换为三次贝塞尔曲线。 * 每个点存储：{x, y, tx, ty} 其中 (tx,ty) 为切线向量（从点指向控制柄末端） */
```

作用：文件头部注释，说明本编辑器的核心特性（多曲线、每点可调斜率）以及数据存储方式。

关键概念：每个控制点除了坐标 还存储一个切线向量，该向量决定曲线在该点的斜率（方向和力度）。

javascript

```
(function(){
```

作用：立即执行函数表达式（IIFE），创建一个独立的作用域，避免全局变量污染。所有变量和函数都封装在此函数内部。

2. DOM 元素获取

javascript

```
const canvas = document.getElementById('curveCanvas');
const ctx = canvas.getContext('2d');
const curveSelect = document.getElementById('curveSelect');
const addCurveBtn = document.getElementById('addCurveBtn');
const delCurveBtn = document.getElementById('delCurveBtn');
const clearPointsBtn = document.getElementById('clearPointsBtn');
const saveBtn = document.getElementById('saveBtn');
```

作用：获取 HTML 页面中的各个元素，以便后续绑定事件或操作。

canvas：画布元素，用于绘图。

ctx：Canvas 2D 上下文，提供绘图 API。

curveselect：下拉框，用于切换当前编辑的曲线。

各个按钮：添加曲线、删除曲线、清空控制点、保存图片。

3. 画布尺寸常量

javascript

```
const width = 900, height = 550;
canvas.width = width;
canvas.height = height;
```

作用：定义画布的实际像素尺寸（900×550），并显式设置到 canvas 元素上，避免 CSS 缩放导致的坐标映射错误。

4. 数据结构与全局状态

javascript

```
let curves = [], // 存储所有曲线对象
currentCurveIndex = 0, // 当前编辑的曲线索引
nextCurveId = 1;
```

curves：数组，每个元素是一个曲线对象（见下文 createCurve）。

currentCurveIndex：当前选中的曲线在 curves 数组中的索引。

nextCurveID：自增 ID，用于为每条曲线生成唯一标识。

javascript

```
// 交互状态
```

```
let dragType = null; // 'point', 'tangent'
let dragTarget = null; // { curvelIdx, pointIdx }
let isDragging = false;
let startMouse = {x: 0, y: 0};
```

拖拽状态变量：

dragType：当前拖拽的类型，是控制点（）还是切线手柄（）。

dragTarget：记录拖拽的目标，包含曲线索引和点索引。

isDragging：是否处于拖拽状态。

startMouse：拖拽起始时的鼠标坐标（本版本中未用于计算位移，但保留以备扩展）。

```
javascript
// 视觉常量
```

```
const POINT_RADIUS=6;
```

```
const TANGENT_HANDLE_RADIUS=5;
```

视觉常量：控制点的圆半径（6px）、切线手柄的圆半径（5px）。

5. 辅助函数

曲线颜色生成

```
javascript
```

```
function getCurveColor(index){
```

```
const hue =(index *137)%360;// 黄金角近似
```

```
return `hsl(${hue}, 65%, 55%)`;}
```

作用：根据曲线索引生成一个较为独特的颜色（HSL 色相环上以 137° 递增，近似黄金角，使相邻曲线颜色差异明显）。

参数：index 曲线的序号。

返回：hsl(hue,65%,55%)格式的颜色字符串。

6. 创建新曲线对象

```
javascript
```

```
function createCurve(name = null){
```

```
const id = nextCurveId++;
```

```
return{
```

```
id: id,
```

```
name: name || `曲线 ${id}`,
```

```
points:[],// 每个元素 {x, y, tx, ty}
```

```
color:getCurveColor(curves.length)};
```

作用：工厂函数，生成一个新的曲线对象。

属性：

id：唯一数字标识。

name：显示名称，若未提供则自动生成“曲线 x”。

points：控制点数组，每个点包含 {x,y,tx,ty}。

color：调用 getCurveColor 基于当前已有曲线数量确定颜色。

7. 向曲线添加控制点

```
javascript
```

```
function addPointToCurve(curve, x, y){
```

```
// 避免过近重叠
```

```
if(curve.points.some(p=> Math.hypot(p.x - x, p.y - y)<POINT_RADIUS*1.5))return false;
```

```
// 默认切线方向：从该点指向右偏下 (dx=30, dy=10) 作为初始，避免零向量
```

```
const defaultTx =30;
```

```
const defaultTy =10,
```

```
curve.points.push({ x, y, tx: defaultTx, ty: defaultTy });
```

```
return true;}
```

作用：为指定曲线添加一个新的控制点。

防重叠：如果新点与已有任意点的欧氏距离小于 POINT_RADIUS*1.5（约9px），则拒绝添加。

默认切线：(30,10)指向右下，保证非零向量，便于显示手柄。

返回值：ture 表示添加成功，false 表示因重叠而失败。

8. 更新下拉菜单

```
javascript
```

```
function updateCurveSelect(){
```

```
curveSelect.innerHTML ='';
```

```
curves.forEach((curve, idx)=>{const option = document.createElement('option'),
```

```
option.value = idx;
```

```

option.textContent = `${curve.name} (${curve.points.length}点);
if(idx === currentCurveIndex) option.selected = true;
curveSelect.appendChild(option);
if(curves.length === 0){
const option = document.createElement('option'),
option.text = 无曲线，请新建;
curveSelect.appendChild(option);}

```

作用：根据curves数组重新构建下拉菜单的选项，并高亮当前曲线。

细节：每个选项显示曲线名称和点数，当前曲线设为 selected。若无曲线则显示提示项。

9. 切换当前曲线

javascript

```

function setCurrentCurve(index){
if(index >= 0 && index < curves.length){
currentCurveIndex = index;
updateCurveSelect();
redraw();}

```

作用：更改当前编辑的曲线索引，同步下拉菜单，并刷新画布。

10. 删除当前曲线

javascript

```

function deleteCurrentCurve(){
if(curves.length <= 1){
alert('至少保留一条曲线');
return;}
curves.splice(currentCurveIndex, 1);
if(currentCurveIndex >= curves.length) currentCurveIndex = curves.length - 1;
updateCurveSelect();
redraw();}

```

作用：删除currentCurveIndex指向的曲线。如果删除后曲线数量为0，则至少保留一条（通过 alert 阻止）。删除后调整当前索引为最后一条，并更新界面。

11. 清空当前曲线的所有点

javascript

```

function clearCurrentCurvePoints(){
if(curves[currentCurveIndex]){
curves[currentCurveIndex].points = [];
redraw();}

```

作用：将当前曲线的points数组置空，然后重绘。

12. 绘图函数

绘制网格

javascript

```

function drawGrid(){
ctx.save();
ctx.strokeStyle = "#e2e8f0";
ctx.lineWidth = 0.8;
const step = 25;
for(let x = step, x < width, x += step){
ctx.beginPath();
ctx.moveTo(x, 0);
ctx.lineTo(x, height);
ctx.stroke();}
for(let y = step, y < height, y += step){

```

```

ctx.beginPath(),
ctx.moveTo(0, y),
ctx.lineTo(width, y);
ctx.stroke();
ctx.restore();

```

作用：在画布上绘制间距25px的浅灰色网格，辅助定位。

优化：使用ctx.save()和ctx.restore()保存/恢复绘图状态，避免影响后续绘制。

13. 绘制单条曲线（核心算法）

```

javascript
function drawCurve(curve){
  const points = curve.points;
  if(points.length < 2) return,
  ctx.save(),
  ctx.strokeStyle = curve.color,
  ctx.lineWidth = 3,
  ctx.lineJoin = "round",
  ctx.lineCap = "round",
  ctx.beginPath();
  // 对于每段 [P_i, P_{i+1}], 使用切线向量构造三次贝塞尔曲线
  for(let i = 0; i < points.length - 1; i++){
    const p0 = points[i];
    const p1 = points[i+1];
    // 贝塞尔控制点公式 (Hermite → Bezier)
    const cp1x = p0.x + p0.tx / 3;
    const cp1y = p0.y + p0.ty / 3;
    const cp2x = p1.x - p1.tx / 3;
    const cp2y = p1.y - p1.ty / 3;
    if(i !== 0) ctx.moveTo(p0.x, p0.y);
    ctx.bezierCurveTo(cp1x, cp1y, cp2x, cp2y, p1.x, p1.y);
  }
  ctx.stroke(),
  ctx.restore();
}

```

作用：根据给定曲线的控制点及切线向量，绘制平滑曲线。

算法原理：在 Hermite 插值中，曲线段从 P0 到 P1，起点切线向量为 T0，终点切线向量为 T1。转换为三次贝塞尔曲线时，两个中间控制点为： $\text{textC1} = P0 + T0 / 3$ $\text{C2} = P1 - T1 / 3$ 这样能保证曲线在端点处的导数连续（一阶连续性）。

除以3的原因：贝塞尔曲线的参数化性质，使得切线长度直接影响曲线形状，除以3后切线向量长度与曲线偏离距离直接相关（更直观）。

遍历：对每一段相邻点调用bezierCurveTo，整条曲线连续。

14. 绘制控制点与切线手柄

```

javascript
function drawHandles(){
  for(let cldx = 0; cldx < curves.length; cldx++){
    const curve = curves[cldx];
    const isCurrent = (cldx === currentCurveIndex);
    for(let i = 0; i < curve.points.length; i++){
      const p = curve.points[i];
      // 绘制切线辅助线（虚线）和手柄端点
      const handleX = p.x + p.tx;
      const handleY = p.y + p.ty,
      ctx.save(),

```

```

ctx.beginPath();
ctx.moveTo(p.x, p.y),
ctx.lineTo(handleX, handleY),
ctx.strokeStyle = isCurrent ? "#c44569" : "#aaa";
ctx.setLineDash([5,5]),
ctx.lineWidth = 1.5;
ctx.stroke();
ctx.setLineDash([]);
// 手柄端点圆
ctx.beginPath();
ctx.arc(handleX, handleY, TANGENT_HANDLE_RADIUS, 0, 2*Math.PI),
ctx.fillStyle = isCurrent ? "#c44569" : "#b0b0b0";
ctx.fill(),
ctx.strokeStyle = "#fff",
ctx.lineWidth = 1;
ctx.stroke();
// 绘制控制点
ctx.beginPath();
ctx.arc(p.x, p.y, POINT_RADIUS, 0, 2*Math.PI);
ctx.fillStyle = isCurrent ? "#ff7e5e" : "#9e9e9e";
ctx.fill(),
ctx.strokeStyle = "#ffffff",
ctx.lineWidth = 2;
ctx.stroke();
// 内圈高光
ctx.beginPath();
ctx.arc(p.x, p.y, POINT_RADIUS-2, 0, 2*Math.PI);
ctx.fillStyle = isCurrent ? "#f9a66c" : "#c0c0c0";
ctx.fill(),
ctx.restore();}}

```

作用：绘制所有曲线的控制点和切线手柄。

颜色区分：当前曲线用鲜艳颜色（橙色控制点、粉色手柄），非当前曲线用灰色，视觉上突出当前编辑的曲线。

切线手柄：绘制从控制点到手柄末端的虚线。手柄末端是一个小圆，拖拽它可改变 (tx,ty)。

控制点：两层圆（外圈和内圈高光），增强立体感。

15. 全局重绘入口

javascript

```

function redraw(){
  ctx.clearRect(0,0, width, height);
  drawGrid();
  for(let curve of curves){
    drawCurve(curve);
  }
  drawHandles();
}

```

作用：清空画布，依次绘制网格、所有曲线、所有控制点/手柄。每次数据变化后调用。

16. 交互处理（坐标转换与命中测试）

获取鼠标在 canvas 上的像素坐标

javascript

```

function getMouseCoords(e){
  const rect = canvas.getBoundingClientRect();
  const scaleX = canvas.width / rect.width;

```

```

const scaleY = canvas.height / rect.height;
let clientX, clientY;
if(e.touches){
  clientX = e.touches[0].clientX;
  clientY = e.touches[0].clientY;}else{
  clientX = e.clientX;
  clientY = e.clientY;}
let x =(clientX - rect.left)* scaleX;
let y =(clientY - rect.top)* scaleY;
x = Math.min(Math.max(0, x), width);
y = Math.min(Math.max(0, y), height);
return{ x, y };}

```

作用：将鼠标或触摸点的屏幕坐标转换为 canvas 画布上的实际像素坐标（考虑 canvas 的 CSS 缩放）

。

支持触摸：检测e.touches以适配移动端。

边界裁剪：确保坐标在[0,width]和[0,height]范围内。

17. 命中测试（检测鼠标下方是什么）

javascript

```

function hitTest(mx, my){
  for(let cldx =0; cldx < curves.length; cldx++){
    const curve = curves[cldx];
    for(let pldx =0; pldx < curve.points.length; pldx++){
      const p = curve.points[pldx];
      const hx = p.x + p.tx;
      const hy = p.y + p.ty;
      const distToHandle = Math.hypot(mx - hx, my - hy);
      if(distToHandle <=TANGENT_HANDLE_RADIUS+5){
        return{type:'tangent',curveldx: cldx,pointldx: pldx };}
      const distToPoint = Math.hypot(mx - p.x, my - p.y);
      if(distToPoint <=POINT_RADIUS+5){
        return{type:'point',curveldx: cldx,pointldx: pldx };}}}
  returnnull;}

```

作用：给定鼠标坐标，判断是否命中了某个控制点或切线手柄。优先检测手柄（因为手柄区域较小，且希望优先处理）。

阈值：半径 + 5px，便于点击。

返回值：若命中，返回包含type\curveldx\pointldx的对象；否则null。

18. 事件处理函数

鼠标/触摸按下

javascript

```

function onMouseDown(e){
  e.preventDefault();
  const{ x, y }=getMouseCoords(e);
  const hit =hitTest(x, y);
  if(hit){
    dragType = hit.type;
    dragTarget ={curveldx: hit.curveldx,pointldx: hit.pointldx };
    sDragging =true;
    startMouse ={ x, y };
    // 如果点击的不是当前曲线，自动切换
    if(hit.curveldx !== currentCurveIndex){

```

```

setCurrentCurve(hit.curveIdx);}
canvas.style.cursor = dragType === 'point' ? 'grabbing' : 'crosshair';
return;}
// 空白处添加控制点到当前曲线
if (curves.length > 0){
const curve = curves[currentCurveIndex];
addPointToCurve(curve, x, y);
redraw();
updateCurveSelect();}}

```

逻辑：

阻止默认事件（防止拖拽时选中页面内容）。

获取鼠标坐标，进行命中测试。

如果命中某个元素：记录拖拽类型、目标、开始拖拽标志。

如果命中的曲线不是当前曲线，自动切换过去。修改鼠标光标样式。如果未命中且存在曲线（即画布空白处），则在当前曲线上添加一个新控制点。

19. 鼠标/触摸移动

```

javascript function onMouseMove(e){
if (!isDragging || !dragTarget) return;
e.preventDefault();
const { x, y } = getMouseCoords(e);
const curve = curves[dragTarget.curveIdx];
if (!curve) return;
const point = curve.points[dragTarget.pointIdx];
if (dragType === 'point'){
point.x = x;
point.y = y;
redraw();} else if (dragType === 'tangent'){
let newTx = x - point.x;
let newTy = y - point.y;
if (Math.hypot(newTx, newTy) < 1){
newTx = 1; newTy = 0;}
point.tx = newTx;
point.ty = newTy;
redraw();}
canvas.style.cursor = 'grabbing';
}

```

逻辑：

仅在拖拽状态时执行。

如果是拖拽控制点：直接更新点的坐标，重绘。

如果是拖拽切线手柄：计算新切线向量 = (鼠标坐标 - 点坐标)，并确保长度至少为1（避免零向量导致曲线异常），然后更新(tx,ty),重绘。

20. 鼠标/触摸释放

```

javascript
function onMouseUp(e){
if (isDragging){
isDragging = false;
dragType = null;
dragTarget = null;
canvas.style.cursor = 'crosshair';
redraw();}}

```

作用：结束拖拽，重置相关状态，恢复光标样式。

21. 鼠标离开画布

javascript

```
function onMouseLeave(e){  
  if(isDragging){  
    isDragging = false;  
    dragType = null;  
    dragTarget = null;  
    canvas.style.cursor = 'crosshair';  
    redraw();  
  }  
}
```

作用：防止当鼠标移出画布时仍然保持拖拽状态，造成 UI 错误。

22. 按钮功能实现

添加新曲线

javascript

```
function addNewCurve(){  
  const newCurve = createCurve(),  
    curves.push(newCurve),  
    currentCurveIndex = curves.length - 1;  
  updateCurveSelect();  
  redraw();  
}
```

作用：创建一个空白曲线，添加到curves数组末尾，并自动切换到该曲线。

23. 保存为图片

javascript

```
function saveAsImage(){  
  const link = document.createElement('a');  
  link.download = 'curve_editor.png';  
  link.href = canvas.toDataURL('image/png');  
  link.click();  
}
```

作用：利用 Canvas API 将当前画布内容导出为 PNG 图片，并触发浏览器下载。

24. 事件绑定与初始化

javascript

```
function initEvents(){  
  canvas.addEventListener('mousedown', onMouseDown);  
  window.addEventListener('mousemove', onMouseMove);  
  window.addEventListener('mouseup', onMouseUp);  
  canvas.addEventListener('mouseleave', onMouseLeave);  
  // 触摸事件简单支持  
  canvas.addEventListener('touchstart', onMouseDown);  
  window.addEventListener('touchmove', onMouseMove);  
  window.addEventListener('touchend', onMouseUp);  
  canvas.addEventListener('touchcancel', onMouseLeave);  
  curveSelect.addEventListener('change', (e) => {  
    setCurrentCurve(parseInt(e.target.value));  
  });  
  addCurveBtn.addEventListener('click', addNewCurve);  
  delCurveBtn.addEventListener('click', deleteCurrentCurve);  
  clearPointsBtn.addEventListener('click', clearCurrentCurvePoints);  
  saveBtn.addEventListener('click', saveAsImage);  
}
```

作用：为 canvas 及窗口绑定鼠标/触摸事件，为各个按钮绑定点击处理函数。

注意：mousemove和mouseup绑定在window上，确保即使鼠标移出画布也能正确结束拖拽。

25. 初始化入口

javascript

```
function init(){  
  // 创建第一条示例曲线，带几个控制点并预设有趣的切线方向  
  const demoCurve = createCurve('示例曲线');  
  const pointsDemo =[  
    {x:150,y:400,tx:40,ty:-20},  
    {x:320,y:250,tx:30,ty:50},  
    {x:500,y:300,tx:-20,ty:-30},  
    {x:680,y:450,tx:-40,ty:20}];  
  for(let p of pointsDemo){  
    demoCurve.points.push({...p});  
    curves.push(demoCurve),  
    currentCurveIndex =0;  
    updateCurveSelect();  
    initEvents();  
    redraw();  
  }  
  init();  
}
```

作用：页面加载时创建一条示例曲线，包含4个控制点及其自定义切线向量（使曲线初始就有有趣的形状）。然后初始化下拉菜单、绑定事件、首次绘制。

26. 结束 IIFE

javascript

```
})();
```

作用：结束立即执行函数，所有内部变量不会泄漏到全局作用域。

试用水印

